

Module -5

RBT Level: L1, L2, L3

I/O Port Interfacing & Programming: I/O Programming in 8051 C, LCD interfacing, DAC 0808 Interfacing, ADC 0804 interfacing, Stepper motor interfacing, DC motor control & Pulse Width Modulation (PWM) using C only. (Text book 1- 7.2, 12.1, 13.1, 13.2, 17.2, 17.3)

Interfacing 8051 to ADC-0804



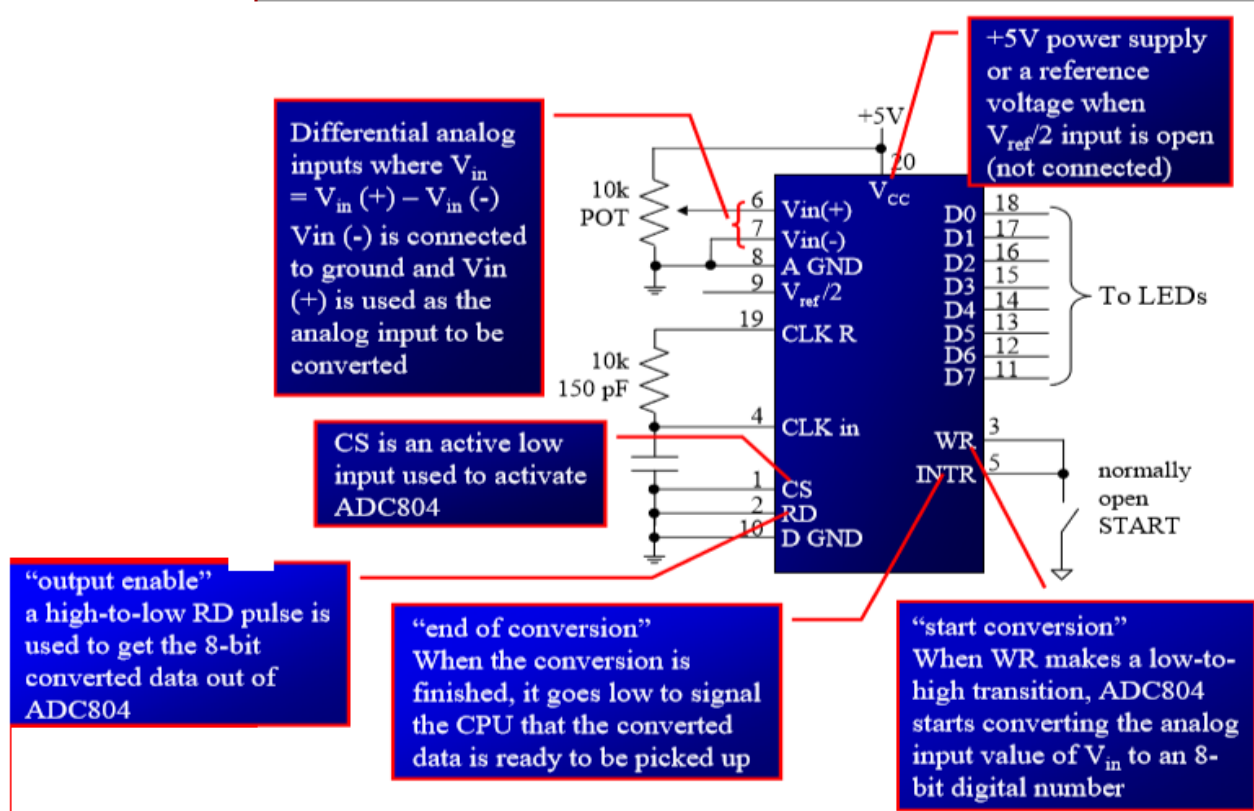
0v – 00h.....5v - ffh

ADCs (analog-to-digital converters) are among the most widely used devices for data acquisition. A physical quantity, like temperature, pressure, humidity, and velocity, etc., is converted to electrical (voltage, current) signals using a device called a transducer, or sensor.

We need an analog-to-digital converter to convert the analog signals to digital numbers, so microcontroller can read them.

ADC804 IC is an analog-to-digital converter. It works with +5 volts and has a resolution of 8 bits. Conversion time is another major factor in judging an ADC. Conversion time is defined as the time it takes the ADC to convert the analog input to a digital (binary) number. *f*

In ADC804 conversion time varies depending on the clocking signals applied to CLK R and CLK IN pins, but it cannot be faster than 110 μ s.



$V_{ref}/2$:

It is used for the reference voltage.

If this pin is open (not connected), the analog input voltage is in the range of 0 to 5 volts (the same as the V_{CC} pin). If the analog input range needs to be 0 to 4 volts, $V_{ref}/2$ is connected to 2 volts.

$V_{ref}/2$ Relation to V_{in} Range

$V_{ref}/2(V)$	$V_{in}(V)$	Step Size (mV)
Not connected*	0 to 5	$5/256=19.53$
2.0	0 to 4	$4/255=15.62$
1.5	0 to 3	$3/256=11.71$
1.28	0 to 2.56	$2.56/256=10$
1.0	0 to 2	$2/256=7.81$
0.5	0 to 1	$1/256=3.90$

Step size is the smallest change can be discerned by an ADC

CLK IN and CLK R

CLK IN is an input pin connected to an external clock source. To use the internal clock generator (also called self-clocking), CLK IN and CLK R pins are connected to a capacitor and a resistor, and the clock frequency is determined by

$$f = 1 / 1.1RC$$

Typical values are $R = 10K$ ohms and $C = 150$ pF. We get $f = 606$ kHz and the conversion time is $110 \mu s$.

D0-D7:

The digital data output pins. These are tri-state buffered. The converted data is accessed only when

$CS = 0$ and RD is forced low.

To calculate the output voltage, use the following formula

$D_{out} = V_{in} / \text{step size}$

D_{out} = digital data output (in decimal), V_{in} = analog voltage, and step size (resolution) is the smallest change.

Analog ground (AGND) and digital ground (GND)

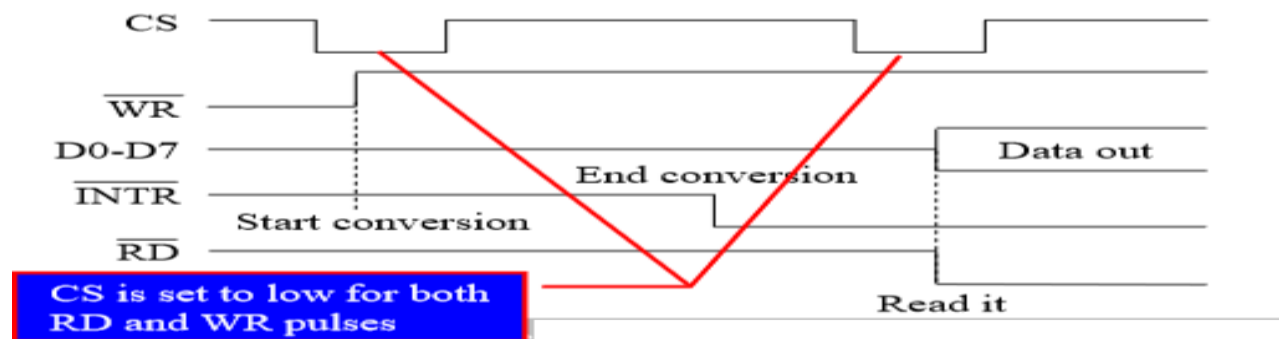
Analog ground is connected to the ground of the analog V_{in} . Digital ground is connected to the ground of the V_{cc} pin. To isolate the analog V_{in} signal from transient voltages caused by digital switching of the output D0 –D7. This contributes to the accuracy of the digital data output.

The following steps must be followed for data conversion by the ADC804 chip.

Make $CS = 0$ and send a low-to-high pulse to pin WR to start conversion.

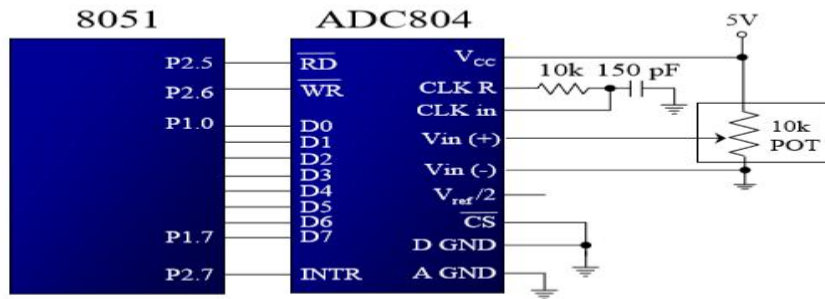
Keep monitoring the $INTR$ pin. If $INTR$ is low, the conversion is finished, If the $INTR$ is high, keep polling until it goes low

After the $INTR$ has become low, we make $CS = 0$ and send a high-to-low pulse to the RD pin to get the data out of the ADC804.



Interfacing ADC0804 to 8051

Interface ADC804 to the 8051 and Write a program to convert an analog input into digital value and store the result in register A. Then call a hex-to ASCII conversion and data display subroutines. Do this continuously.

8051 Connection to ADC804 with Self-Clocking**Program**

```

;p2.6=WR (start conversion needs to L-to-H pulse)
;p2.7 When low, end-of-conversion)
;p2.5=RD (a H-to-L will read the data from ADC chip)
;p1.0 -P1.7= D0 -D7 of the ADC804 ;
    ORG 00H
    MOV  P1,#0FFH    ;make P1 = input
BACK: CLR  P2.6      ;WR = 0
      SETB P2.6      ;WR = 1 L-to-H to start conversion
HERE: JB  P2.7,HERE  ;wait for end of conversion
      CLR  P2.5      ;conversion finished, enable RD
      MOV  A,P1      ;read the data
      ACALL CONVERSION ;hex-to-ASCII conversion
      ACALL DATA_DISPLAY;display the data
      SETB p2.5      ;make RD=1 for next round
      SJMP BACK

```

C Program

```

#include <reg51.h>

void delay (unsigned int count)
{
    unsigned int i, j;
    for (i=0;i<count;i++)
        for (j=0;j<1000;j++);
}

```

```
void main (void)
{
P1=0xff;
While(1)
{
P2.6=0;
Delay(1)
P2.6=1;
while (P2.7==1);
P2.5=0;
ACC=P1;
P2.5=1;
}
}
```

INTERFACING OF LCD

LCD is finding widespread use replacing LEDs .

- (i) The declining prices of LCD
- (ii) The ability to display numbers, characters, and graphics
- (iii) Incorporation of a refreshing controller into the LCD, thereby relieving the CPU of the task of refreshing the LCD
- (iv) Ease of programming for characters and graphics

PIN DESCRIPTIONS OF LCD CONTROLLER

Pin	Symbol	I/O	Descriptions
1	VSS	--	Ground
2	VCC	--	+5V power supply
3	VEE	--	Power supply to control contrast
4	RS	I	RS=0 to select command register, RS=1 to select data register
5	R/W	I	R/W=0 for write, R/W=1 for read
6	E	I/O	Enable
7	DB0	I/O	The 8-bit data bus
8	DB1	I/O	The 8-bit data bus
9	DB2	I/O	The 8-bit data bus
10	DB3	I/O	The 8-bit data bus
11	DB4	I/O	The 8-bit data bus
12	DB5	I/O	The 8-bit data bus
13	DB6	I/O	The 8-bit data bus
14	DB7	I/O	The 8-bit data bus

used by the LCD to latch information presented to its data bus

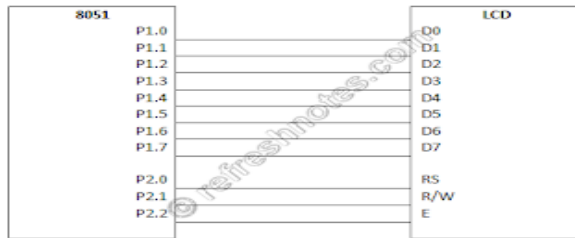
LCD Command Codes

Code (Hex)	Command to LCD Instruction Register
1	Clear display screen
2	Return home
4	Decrement cursor (shift cursor to left)
6	Increment cursor (shift cursor to right)
5	Shift display right
7	Shift display left
8	Display off, cursor off
A	Display off, cursor on
C	Display on, cursor off
E	Display on, cursor blinking
F	Display on, cursor blinking
10	Shift cursor position to left
14	Shift cursor position to right
18	Shift the entire display to the left
1C	Shift the entire display to the right
80	Force cursor to beginning to 1st line
C0	Force cursor to beginning to 2nd line
38	2 lines and 5x7 matrix

To send any of the commands to the LCD, make pin RS=0. Then send a high-to-low pulse to the E pin to enable the internal latch of the LCD.

To send any of the data to the LCD, make pin RS=1. Then send a high-to-low pulse to the E pin to enable the internal latch of the LCD.

Interface an LCD to 8051, and write an ALP to display the message on LCD.



;calls a time delay before sending next data/command

;P1.0-P1.7 are connected to LCD data pins D0-D7

;P2.0 is connected to RS pin of LCD

;P2.1 is connected to R/W pin of LCD

;P2.2 is connected to E pin of LCD

Steps to write the command on Command Register

1. Place the command hex value on P1 to make the command available on data lines of LCD controller //MOV P1, #01H
2. Select command Register by setting RS=0// CLR P2.0
3. To perform write operation, SET R/W=0 //CLR P2.1
4. To write the command value on the command Register apply a High to Low pulse on E pin //SETB P2.2
CLR P2.2

Steps to write the ASCII Data on Data Register to display the character on LCD

1. Place the ASCII value of the character to be displayed on P1 to make the ASCII data available on data lines of LCD controller //MOV P1, #31H
2. Select DATA Register by setting RS=1 // SETB P2.0
3. To perform write operation, SET R/W=0 //CLR P2.1
4. To write the ASCII value on the DATA Register apply a High to Low pulse on E pin // SETB P2.2
CLR P2.2
5. Then the character will be displayed on LCD

PROGRAM

```

ORG 00H
MOV A,#38H;INIT. LCD 2 LINES, 5X7 MATRIX
ACALL COMNWRT ;call command subroutine
ACALL DELAY ;give LCD some time
MOV A,#0EH;display on, cursor on
ACALL COMNWRT ;call command subroutine
  
```

```
ACALL DELAY ;give LCD some time
MOV A,#01;clear LCD
ACALL COMNWRT ;call command subroutine
ACALL DELAY ;give LCD some time
MOV A,#06H;shift cursor right
ACALL COMNWRT ;call command subroutine
ACALL DELAY ;give LCD some time
MOV A,#80H;cursor at line 1, pos. 1
ACALL COMNWRT ;call command subroutine
MOV A, #'S'
ACALL DATAWRT
ACALL DELAY
MOV A, #'C'
ACALL DATAWRT
ACALL DELAY
MOV A, #'E'
ACALL DATAWRT
ACALL DELAY
STOP: SJMP STOP ;stay here
COMNWRT: ;send command to LCD
MOV P1,A ;copy regA to port 1
CLR P2.0 ;RS=0 for command
CLR P2.1 ;R/W=0 for write
SETB P2.2 ;E=1 for high pulse
ACALL DELAY ;give LCD some time
CLR P2.2 ;E=0 for H-to-L pulse
RET
DATAWRT: ;write data to LCD
MOV P1,A ;copy regA to port 1
SETB P2.0 ;RS=1 for data
CLR P2.1 ;R/W=0 for write
SETB P2.2 ;E=1 for high pulse
ACALL DELAY ;give LCD some time
CLR P2.2 ;E=0 for H-to-L pulse
```

RET

DELAY: MOV R3,#50 ;50 or higher for fast CPUs

HERE2: MOV R4,#255 ;R4 = 255

HERE: DJNZ R4,HERE ;stay until R4 becomes 0

DJNZ R3,HERE2

RET

END

Interface an LCD to 8051 and write An embedded C program to display the message “GOODLUCK” on 6th position of the first line of LCD

```
#include <reg51.h>
```

```
Sfr ldata= 0x90; //P1=LCD data pins
```

```
Sbit rs= P2^0;
```

```
Sbit rw= P2^1;
```

```
Sbit en = P2^2;
```

```
Void lcdcmd ( unsignedchar value)
```

```
{
```

```
ldata= value; //put the value on the pins
```

```
rs= 0;
```

```
rw= 0;
```

```
en = 1; //strobe the enable pin
```

```
Delay(1);
```

```
en = 0;
```

```
return;
```

```
}
```

```
voidlcddata(unsignedchar value)
```

```
{
```

```
ldata= value; //put the value on the pins
```

```
rs= 1;
```

```
rw= 0;
```

```
en = 1; //strobe the enable pin
```

```
Delay(1);
```

```
en = 0;
```

```
return;
```

```
}
```

```
void Delay(unsigned int itime)
{
    unsignedint i, j;
    for(i=0;i<itime;i++)
    for(j=0;j<1275;j++);
}

void main()
{
    lcdcmd(0x38);
    Delay(100);
    lcdcmd(0x0E);
    Delay(100);
    lcdcmd(0x01);
    Delay(100);
    lcdcmd(0x06);
    Delay(100);
    lcdcmd(0x86); //line 1, position 6
    Delay(100);
    lcddata('G');
    Delay(100);
    lcddata('O');
    Delay(100);
    lcddata('O');
    Delay(100);
    lcddata('D');
    Delay(100);
    lcddata('L');
    Delay(100);
    lcddata('U');
    Delay(100);
    lcddata('C');
    Delay(100);
    lcddata('K');
    Delay(100);
```

}

INTERFACING STEPPER MOTOR**Stepper Motor:**

A stepper motor is a device that translates electrical pulses into mechanical movement in steps of fixed step angle.

- The stepper motor rotates in steps in response to the applied signals.
- It is mainly used for position control.
- It is used in disk drives, dot matrix printers, plotters and robotics and process control circuits.

Structure

Stepper motors have a permanent magnet called rotor (also called the shaft) surrounded by a stator. The most common stepper motors have four stator windings that are paired with a center-tap. This type of stepper motor is commonly referred to as a four-phase or unipolar stepper motor. The center tap allows a change of current direction in each of two coils when a winding is grounded, thereby resulting in a polarity change of the stator.

Interfacing Even a small stepper motor require a current of 400 mA for its operation. But the ports of the microcontroller cannot source this much amount of current. If such a motor is directly connected to the microprocessor/microcontroller ports, the motor may draw large current from the ports and damage it. So a suitable driver circuit is used with the microprocessor/microcontroller to operate the motor.

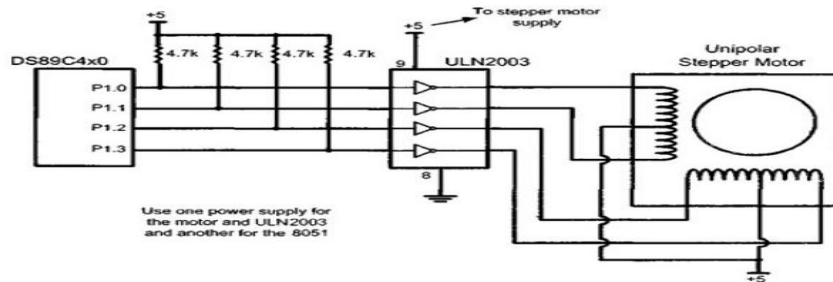
Motor Driver Circuit (ULN2003)

Stepper motor driver circuits are available readily in the form of ICs. ULN2003 is one such driver IC which is a High-Voltage High-Current Darlington transistor array and can give a current of 500mA. This current is sufficient to drive a small stepper motor. Internally, it has protection diodes used to protect the motor from damage due to back emf and large eddy currents. So, this ULN2003 is used as a driver to interface the stepper motor to the microcontroller.

Operation

The important parameter of a stepper motor is the step angle. It is the minimum angle through which the motor rotates in response to each excitation pulse. In a four phase motor if there are 200 steps in one complete rotation then then the step angle is $360/200 = 1.8^\circ$. So to rotate the stepper motor we have to apply the excitation pulse.

For this the controller should send a hexa decimal code through one of its ports. The hex code mainly depends on the construction of the stepper motor. So, all the stepper motors do not have the same Hex code for their rotation. (refer the operation manual supplied by the manufacturer.) For example, let us consider the hex code for a stepper motor to rotate in clockwise direction is CCH, 66H, 33H and 99H. This hex code will be applied to the input terminals of the driver through the assembly language program.



Program to rotate the motor in clockwise direction

Main; MOV A, #0CCH ; Code for the Phase 1

Again: MOV P1, A ;

ACALL DELAY ; Delay subroutine

RR A

SJMP Again ; Keep the motor rotating continuously.

DELAY: MOV R4, #0FF H ; Load R4 with FF

MOV R5, # 0FF ; Load R5 with FF

LOOP1: DJNZ R4, LOOP1 ; Decrement R4 until zero,wait

LOOP2: DJNZ R5, LOOP2 ; Decrement R5 until zero,wait

RET ; Return to main program .

END

To rotate the stepper motor in anti-clockwise direction the same code is applied in the reverse order.

Main; MOV A, #0CCH ; Code for the Phase 1

Again: MOV P1, A ;

ACALL DELAY ; Delay subroutine

RL A

SJMP Again ; //Keep the motor rotating continuously.

DELAY: MOV R4, #0FF H ; //Load R4 with FF

MOV R5, # 0FF ;//Load R5 with FF

LOOP1: DJNZ R4, LOOP1 ; //Decrement R4 until zero,wait

LOOP2: DJNZ R5, LOOP2 ; //Decrement R5 until zero,wait

RET ; //Return to main program .

END

Program to rotate the motor in one clockwise direction and one anticlockwise direction

Main; MOV R2, #200

```

        MOV A, #88H ;           //Code for clockwise rotation
Again: MOV P1, A ;
        ACALL DELAY ;           //Delay subroutine
        RR A
        DJNZ R2, Again ;        //Keep the motor rotating continuously.
        MOV R2, #200
        MOV A, #11H ;           //Code for anticlockwise rotation
Again: MOV P1, A ;
        ACALL DELAY ;           //Delay subroutine
        RL A
        DJNZ R2, Again ;        //Keep the motor rotating continuously.
DELAY: MOV R4, #0FF H ;         //Load R4 with FF
        MOV R5, # 0FF ;         //Load R5 with FF
        LOOP1: DJNZ R4, LOOP1 ;  //Decrement R4 until zero,wait
        LOOP2: DJNZ R5, LOOP2 ;  //Decrement R5 until zero,wait
        RET ;                    //Return to main program .
END

```

C Program

```

# include <reg51.h>
void Delay(unsigned int itime)
{
    unsignedint i, j;
    for(i=0;i<itime;i++)
    for(j=0;j<1275;j++);
}
void main( )
(
    unsigned int stepval;
    unsigned int k,z;
    for(z=0;z<50;z++)
    {
        stepval=0x08;
        for (k=0;k<4;k++)
        {

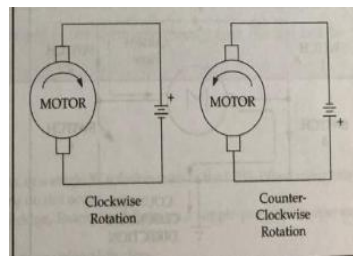
```

```
P1=stepval;
stepval=stepval>>1;
}
}
for(z=0;z<50;z++)
{
stepval=0x01;
for (k=0;k<4;k++)
{
P1=stepval;
stepval=stepval<<1;
}
}
}
```

DC Motor Interfacing

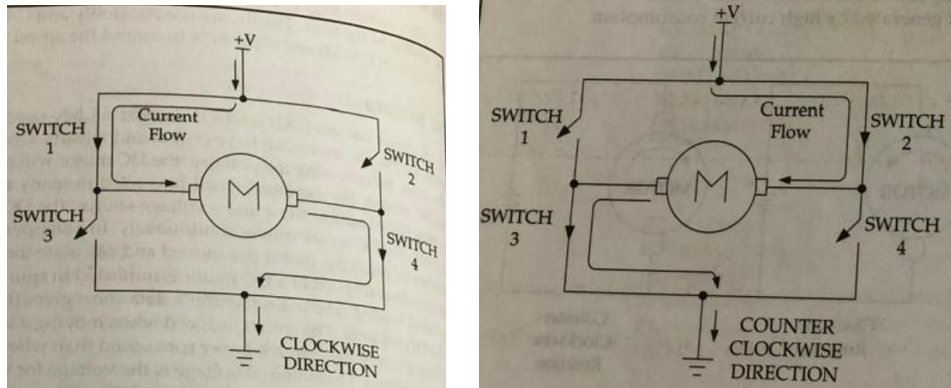
DC motor converts electrical energy in the form of Direct Current into mechanical energy.

- In the case of the motor, the mechanical energy produced is in the form of a rotational movement of the motor shaft.
- Dc motor has 2 leads, + lead and – lead. Connecting these leads to the DC power source moves the motor in one direction.
- The direction of rotation of the shaft of the motor can be reversed by reversing the direction of Direct Current through the motor by connecting the leads to the DC power source in reverse.

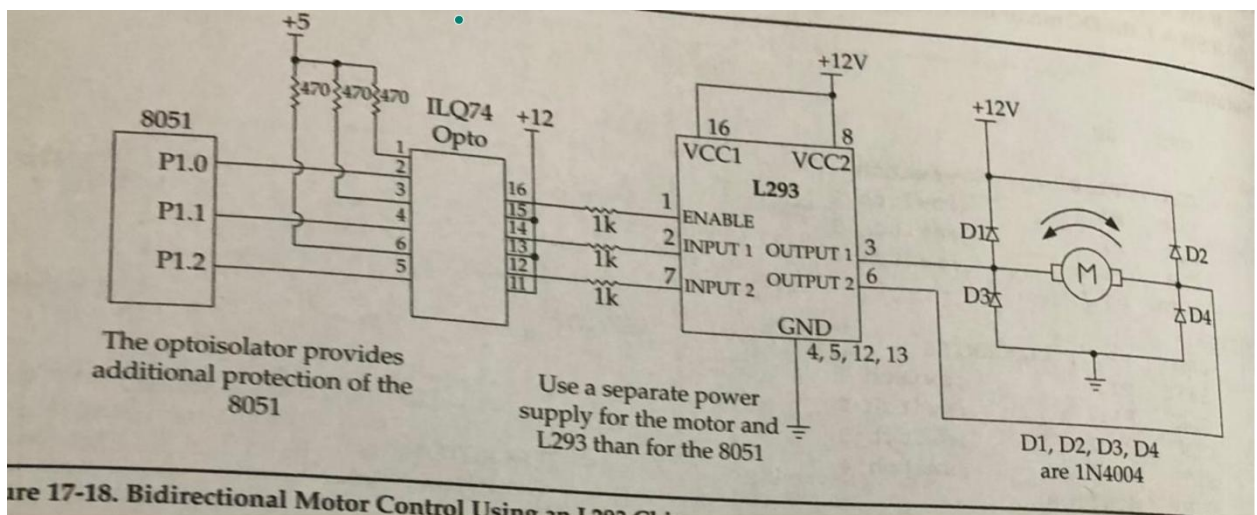


- The motor can be rotated at a certain speed by applying a fixed voltage to it. If the voltage varies, the speed of the motor varies.
- Thus, the DC motor speed can be controlled by applying varying DC voltage; whereas the direction of rotation of the motor can be changed by reversing the direction of current through it.
- For applying varying voltage, we can make use of the PWM technique.
- For reversing the current, we can make use of an H-Bridge circuit or motor driver ICs that employ the H-Bridge technique or any other mechanisms.

4 switches are connected to DC motor in the form of H Bridge to control the direction of the current. Switch 1 and switch 4 are closed to rotate the DC motor in clockwise direction. Switch 2 and switch 3 are closed to rotate the DC motor in counter clockwise direction.



- L293D motor driver IC has been used to control the DC motor movement in both directions. It has an in-built H-bridge motor drive.



A switch SW is connected to P2.7. Write a program to monitor the status of the switch SW and do the following

- If SW=0, DC motor moves clockwise
- If SW=1, DC motor moves counterclockwise

Program

```
ORG 00H
CLR P1.0
CLR P1.1
CLR P1.2
SETB P2.7
```

Monitor:

SETB P1.0
JB P2.7 Counterclockwise

Clockwise:

SETB P1.1
CLR P1.2

Counterclockwise:

CLR P1.1
SETB P1.2
SJMP Monitor
END

C Program

```
# include <reg51.h>
sbit SW = P2^7;
void main( )
{
P2.7=1;
P1.0=1;
P1.1=0;
P1.2=0;
if (SW==1)
{
P1.1=0;
P1.2=1;
}
else
{
P1.1=1;
P1.2=0;
}
}
```

DAC (0808) INTERFACING

Digital to Analog converter (DAC) is a device to convert digital inputs to analog output. Generally there are two different techniques [(i) Binary weighted (ii) R to 2R] available to convert digital inputs to analog. DAC 0808 IC uses R to 2R conversion technique since it can achieve a high degree of precision.

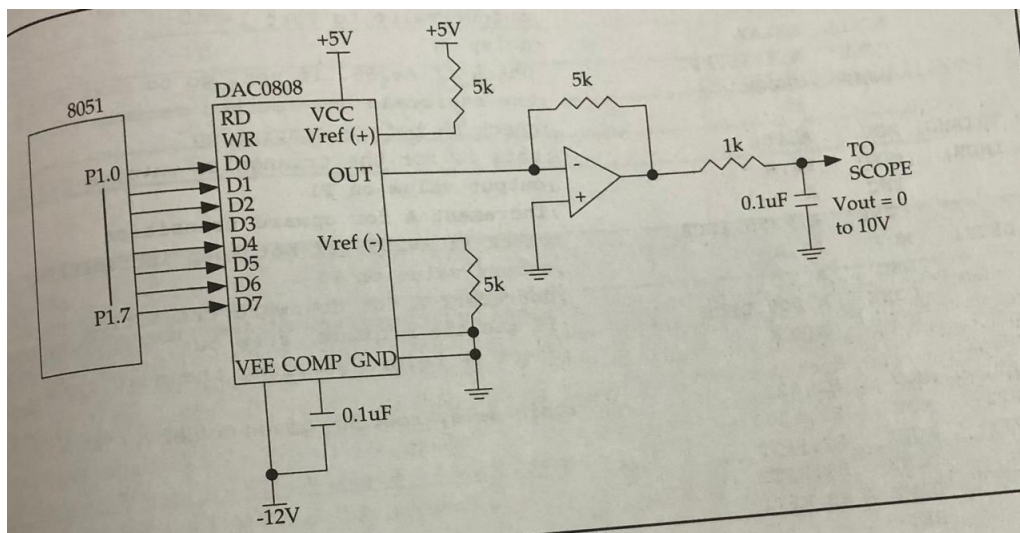
The first criterion for judging a DAC is its resolution in terms of binary inputs. Generally the common ones are 8,10 and 12 bits. The number of data bit inputs (n)

decides the resolution of DAC since the number of analog output levels is equal to 2^n . Therefore an 8 input DAC such as the DAC 0808 provides 256 discrete voltage output.

In DAC 0808, the digital inputs are converted to current (I_{out}) and by connecting a resistor to the I_{out} pin convert the result to voltage. The total current provided by the I_{out} pin is a function of reference current (I_{ref}) and the binary values at the inputs D0 to D7.

$$I_{out} = I_{ref} \left(\frac{D_7}{2} + \frac{D_6}{4} + \frac{D_5}{8} + \frac{D_4}{16} + \frac{D_3}{32} + \frac{D_2}{64} + \frac{D_1}{128} + \frac{D_0}{256} \right)$$

Where D0 is the LSB and D7 is the MSB. I_{ref} is the input current applied to the input pin. I_{ref} current is generally set to 2 mA. This reference current is generated using standard 5V power supply and 1K and 1.5K standard resistors.



The maximum amplitude of analog output voltage will be obtained for maximum digital input FFh

$$I_{out} = 2\text{mA} \times 255/256 = 1.99\text{mA}$$

$$V_{out} = 1.99\text{mA} \times 5\text{K} = 9.96\text{V}$$

Interface DAC 0808 to 8051 and write a program to generate a ramp waveform of maximum amplitude of 10V.

```

ORG 00H
MOV A, #00H
Back: MOV P1, A
      INC A
      SJMP back
      END

```

Square Waveform of 10V amplitude

```
ORG 00h
Again: MOV P1, #00H
      ACALL DELAY
      MOV P1, #0FFH
      ACALL DELAY
      SJMP Again
DELAY:
      MOV R2, #0FFH
L2:    MOV R1, #0FFH
L1:    DJNZ R1, L1
      DJNZ R2, L2
      RET
      END
```

Triangular waveform of 10V amplitude

```
ORG 00h
Again:
      MOV A, #00H
L1:    MOV P1, A
      INC A
      CJNE A, #0FFH, L1
L2:    MOV P1, A
      DEC A
      CJNE A, #00H, L2
      SJMP Again
      END
```

Note: For waveform generation C PROGRAMS refer lab programs